Leveraging Agentic AI Middleware for Intelligent Interaction and Modernization of Legacy Enterprise Applications

M.Z.M.Ayyash
Department of Software Engineering
Sri Lanka Institue of Information Technology
Malabe, Sri Lanka
it22355850@my.sliit.lk

M.A.M.Mufthi
Department of Software Engineering
Sri Lanka Institue of Information Technology
Malabe, Sri Lanka
it22355478@my.sliit.lk

S.M.A.C.M.Rashad
Department of Software Engineering
Sri Lanka Institue of Information Technology
Malabe, Sri Lanka
it22331472@my.sliit.lk

D.N.RAJAPAKSHA
Department of Software Engineering
Sri Lanka Institue of Information Technology
Malabe, Sri Lanka
it22311436@my.sliit.lk

Mr. Vishan Jayasinghearachchi
Department of Software Engineering
Sri Lanka Institue of Information Technology
Malabe, Sri Lanka
vishan.j@sliit.lk

Dr. Dharshana Kasthurirathna
Department of Software Engineering
Sri Lanka Institue of Information Technology
Malabe, Sri Lanka
dharshana.k@sliit.lk

Abstract*—Enterprise organizations across healthcare, finance, and government continue to operate mission-critical software built on decades-old desktop platforms, spending hundreds of billions of dollars annually on maintenance while remaining unable to modernize due to source-code inaccessibility and prohibitive replacement costs. This paper presents LAMA (Legacy Application Modernization Architecture), an agentic AI (Artificial Intelligence) middleware framework that modernizes legacy Windows desktop applications through non-intrusive orchestration, requiring zero modification to the underlying system. LAMA integrates four principal components: an LLM (Large Language Model)-driven frontend generation pipeline that automatically constructs modern web interfaces from legacy screenshots; SOACRS (Self-Optimizing Agent Coordination and Response Service), a self-optimizing coordination microservice that produces adaptive, fallback-aware execution plans using a hybrid rule-based and contextual bandit scoring engine; a stateful multi-agent orchestrator that enforces pre- and post-condition verification across dynamically composed automation workflows; and a computer-vision-augmented RPA (Robotic Process Automation) bot layer combining PyAutoGUI, PyDirectInput, and AutoHotkey for reliable interaction with Win32-era graphical user interfaces (GUIs). The framework was evaluated against a Visual Basic 6 hospital management system running on a Windows XP virtual machine (VM) across 150 executions spanning three clinical workflows. Results demonstrate an overall task success rate of 97.0%, an OCR (Optical Character Recognition) field-level accuracy of 94.1%, and a workflow recovery rate of 98.8%, with a mean end-to-end execution time of 7.8 seconds. All but three failures recovered autonomously through retry and fallback mechanisms, with no execution requiring full compensatory rollback. These findings establish LAMA as a technically viable and practically deployable path toward legacy system modernization in high-continuity, regulated environments.

Keywords—legacy system modernization, robotic process automation, agentic AI, large language model, multi-agent systems, GUI automation, orchestration, computer vision, enterprise middleware

I. INTRODUCTION

Legacy enterprise software remains deeply embedded across healthcare, finance, government, and manufacturing sectors worldwide. A 2023 industry survey estimated that organizations collectively spend over $300 billion annually maintaining legacy systems, with most of that expenditure directed at preventing failures rather than enabling innovation [1]. Despite this cost, full system replacement remains out of reach for most organizations — developers rarely hand over source code to clients, making recreation from scratch both technically difficult and financially prohibitive. The systems, however outdated their interfaces, continue to perform their core functions correctly.

Legacy desktop applications are typically tied to specific operating systems and hardware, blocking remote access and mobile use. In healthcare in particular, staff must physically access dedicated machines to perform registration, scheduling, or billing tasks — creating bottlenecks that slow care delivery and increase error rates.

RPA (Robotic Process Automation) offers a different path. RPA bots interact with applications through the same graphical user interface (GUI) a human would use, requiring no access to source code or internal application programming interfaces (APIs). However, traditional RPA is fragile: hard-coded sequences break when screen layouts change, and there is no capacity to handle variable user input intelligently.

This paper addresses that fragility by introducing an AI-driven orchestration layer above the RPA execution stack. Users interact through a modern interface generated from the legacy system's UI (User Interface). Inputs are passed to a planning module that identifies the required bots and sequence, which an orchestration module then executes directly within the legacy application — with zero modification to the underlying system.

The specific contributions of this work are:

- A hybrid AI-rule execution planning architecture combining LLM flexibility with rule-based determinism to generate reliable, fallback-aware GUI action plans.
- SOACRS, a lightweight Node.js microservice implementing a contextual-bandit-scored tool selection engine with explicit state-machine transitions for success, failure, and timeout.

- A stateful multi-agent orchestration engine that enforces pre- and post-condition verification at each step, coordinating a pool of specialized RPA bots via a DAG (Directed Acyclic Graph)-based execution model and plug-and-play bot registry requiring no middleware restart.
- An empirical evaluation against a Visual Basic 6 Hospital Management System under Windows XP, demonstrating 97.0% task success and 98.8% recovery across 150 trials with zero source-code modification.

II. Related Work

A. Legacy System Modernization

Bennett [2] established the foundational challenge: legacy systems become harder to replace precisely because they succeed — embedding deeply into workflows over time. Kazman et al. [5] developed architecture reconstruction techniques that identify hidden dependencies without full re-engineering, while Perez-Castillo et al. [6] demonstrated screen-scraping as a viable non-intrusive extension mechanism for mainframe systems in regulated environments. Market analyses show that modernization projects average 16 months, cost $1.5 million, and fail or are abandoned 80% of the time, making non-intrusive alternatives commercially compelling.

B. Robotic Process Automation

Lacity and Willcocks [3] confirmed RPA's value for rule-based GUI tasks while identifying selector fragility as the dominant long-term challenge. Ribeiro et al. [7] reviewed 52 studies and found that AI-enhanced RPA significantly improves UI-drift resilience but that existing implementations still lack higher-level coordination layers. El-Gharib et al. [8] showed process mining can identify optimal automation points, improving RPA deployment efficiency. Oberhauser and Pogolski [4] proposed partially self-healing RPA for minor interface changes, but without multi-agent coordination or API exposure capabilities.

C. AI-Driven GUI Understanding

Beltramelli's pix2code [9] demonstrated 77% accuracy generating UI code from screenshots using CNN (Convolutional Neural Network)-LSTM architectures, establishing the feasibility of semantic GUI analysis without framework access. The RICO dataset [10] provided large-scale UI training data. OmniParser [11] advanced the state of the art by parsing screenshots into structured element trees for reliable action grounding. UI-TARS [12] demonstrated autonomous GUI agents operating through native interfaces via visual understanding alone — the closest prior work to the proposed RPA connector, though operating as a standalone agent without coordination or API exposure.

D. Multi-Agent Coordination

Smith's Contract Net Protocol [13] established capability-based decentralized task allocation. Nii's Blackboard Architecture [14] introduced shared-memory opportunistic coordination, both adapted in the proposed Orchestrator. BDI (Belief-Desire-Intention) models [15], [16], [17], [18] formalized goal-directed planning with dynamic revision — directly informing SOACRS's pre/post-condition planning. Li et al. [19] introduced contextual bandits for online adaptive agent selection, balancing exploration and exploitation based on real-time context features. AgentOrchestra [20] and DynTaskMAS [21] demonstrated hierarchical and asynchronous orchestration for complex MAS (Multi-Agent System) environments, validating the DAG-based execution model.

E. Reliability and Observability

Garcia-Molina and Salem's Sagas pattern [22] addresses long-running transaction recovery through atomic sub-transactions with compensatory actions — directly applied to SOACRS plan construction. Sigelman et al.'s Dapper [23] provides the distributed tracing foundation ensuring auditability of all delegation decisions and execution outcomes in the proposed observability stack.

F. Research Gap

No existing system combines automated UI generation from screenshots, BDI-inspired adaptive planning with contextual bandit delegation, DAG-based orchestration with compensation, CNN-driven adaptive RPA execution with digital-twin validation, and REST (Representational State Transfer)/GraphQL API exposure — all without legacy source-code access. LAMA was built specifically to fill this compound gap.

III. System Architecture And Methodology

LAMA modernizes legacy desktop applications without modifying their source code. The framework begins when an administrator captures a screenshot of the legacy application's active interface. An LLM processes this image to extract a structured JSON (JavaScript Object Notation) description of the screen's components. That JSON feeds a code-generation pipeline producing a modern React web interface, which users interact with from any device or operating system. User inputs travel via WebSocket to the SOACRS planning service, which generates an ordered execution plan. The plan is handed to the orchestrator, which dispatches RPA bots to execute it against the legacy system running in a background VM. Fig. 1 illustrates the high-level system architecture. Fig. 2 shows the execution flowchart.

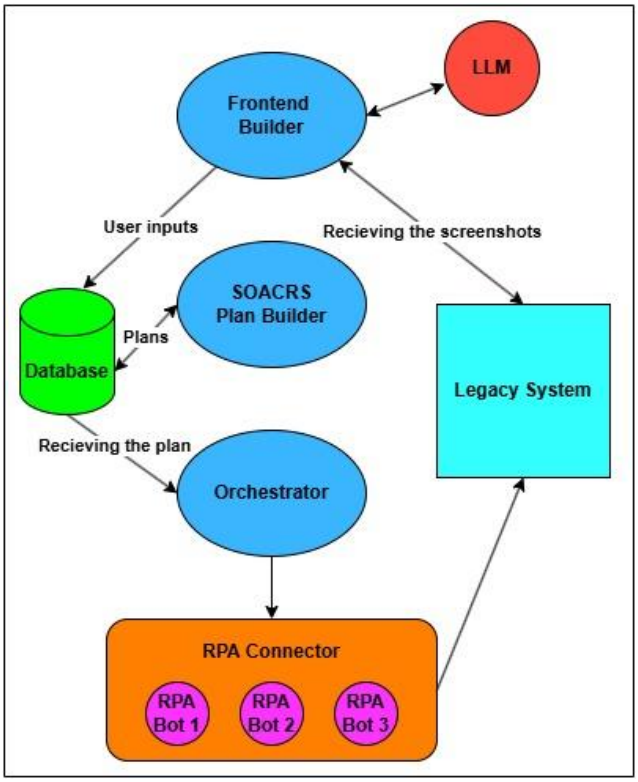

Figure 1 - High level system Architecture diagram

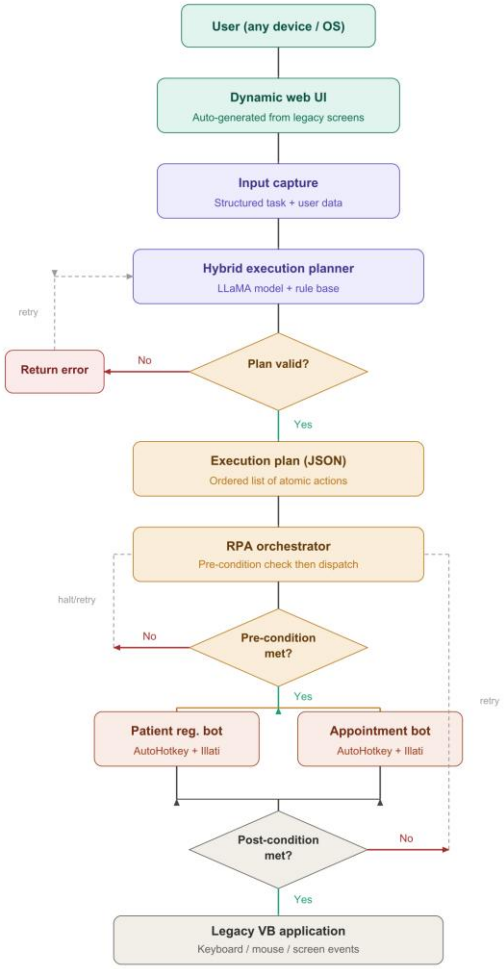

Figure 2 System Flowchart

A. *Dynamic Modern UI Generation from Legacy Interfaces*

The Frontend Builder eliminates the accessibility barrier of legacy systems by auto-generating a full modern web interface from screenshots, enabling remote access from any device without requiring local OS (Operating System) configuration or legacy software installation.

Each target legacy screen is captured as a PNG (Portable Network Graphics) file via PyAutoGUI. The image is submitted to an LLM (Meta LLaMA 3.1 70B or GPT-4V) with a structured prompt requesting JSON extraction across five element categories: form fields (with inferred labels, data types, and validation constraints), action buttons, navigation controls, display elements, and workflow relationships between screens. The LLM infers data types from field labels (e.g., "Date of Birth" maps to a date-picker input type) and flags low-confidence extractions for human review before deployment.

The validated JSON schema is consumed by a code-generation pipeline that maps each element to its React equivalent — controlled inputs with HTML5 types, Zod-validated form schemas, React Router navigation, and sortable data tables. Styling uses Tailwind CSS (Cascading Style Sheets) following WCAG (Web Content Accessibility Guidelines) 2.1 AA accessibility standards. Every user action triggers a structured JSON event object containing the action type, field values, a session token, and a trace UUID (Universally Unique Identifier), transmitted via Secure WebSocket to SOACRS. Real-time status updates are pushed back to the interface as the Orchestrator executes the corresponding RPA workflow.

B. *Soacrs – The Plan Builder*

SOACRS is a lightweight Node.js microservice that sits between the frontend and the orchestrator. Its role is narrow but critical: it takes a structured task description as input and produces a complete, failure-aware routing plan as output. It does not execute anything itself.

At the center of SOACRS is a scoring function that evaluates each registered tool against four criteria for a given task request. The composite score for a candidate tool t is defined as:

$$S(t) = w_1 \cdot C(t) + w_2 \cdot R(t) + w_3 \cdot U(t) - w_4 \cdot K(t) \quad (1)$$

where C(t) is a binary capability-match indicator (1 if the tool declares support for the required task type, 0 otherwise), R(t) is the tool's observed success rate over recent executions tracked as an exponential moving average, U(t) is the normalized mean user-satisfaction rating, and K(t) is the estimated execution cost (normalized and inverted so that lower cost scores higher). The weights w_1–w_4 are configurable per deployment context; defaults are 0.4, 0.3, 0.2, and 0.1 respectively. The highest-scoring eligible tool is selected as the primary; the second-highest becomes the designated fallback.

The resulting execution plan is structured as a state machine. Each action carries explicit transitions for success (advance to the next step), failure (invoke the fallback tool or return an error), and timeout (retry up to a configured limit before escalating). This structure gives the orchestrator

a deterministic path to follow regardless of runtime conditions.

SOACRS is built around a strict separation between domain logic and infrastructure: the scoring and state-machine logic are pure functions with no framework dependencies, while database access and external service calls are isolated behind adapters. This made unit testing straightforward and allowed SOACRS to run in stub mode during frontend development, verifying plan structure without a live legacy environment.

C. Orchestration of Multi-Agent Automation Systems

The orchestrator receives the JSON execution plan from SOACRS and is responsible for running it. Before dispatching any bot, it performs a pre-condition check: it captures a screenshot and checks for expected UI elements to confirm the environment is in the expected state for that step. Only once the pre-condition is satisfied does it dispatch the relevant RPA bot.

After each bot completes its assigned steps, the orchestrator performs a post-condition check using the same screenshot-based verification approach. If the expected outcome is not confirmed — for example, if a newly registered patient does not appear in the patient list — the orchestrator triggers a retry sequence. If the retry also fails, the step is marked as failed and the plan's failure transition is followed.

The execution plan is represented internally as a DAG, where each node is an atomic action and edges represent dependencies. This representation allows the orchestrator to identify and execute independent steps in parallel where the plan permits. A plug-and-play bot registry allows bots to be activated, deactivated, or replaced without restarting the middleware, which substantially shortened the development iteration cycle.

D. RPA Bot Interaction with Legacy Systems

Each bot is a Python process that drives the legacy GUI using a combination of three input strategies: PyAutoGUI for mouse movement and click simulation; PyDirectInput for low-level keyboard injection that bypasses focus-interception issues common in virtualized environments; and AutoHotkey scripts for high-speed, reliable data entry into text fields. This three-strategy hybrid chain was critical — early prototypes using PyAutoGUI alone produced input failures in approximately 12% of trials; adding PyDirectInput reduced this to under 4%; adding AutoHotkey as the tertiary fallback reduced it to under 2%.

Screen coordinates in the VM do not map directly to host screen coordinates, so the system implements relative-coordinate calibration: the bot calculates element positions as fractions of the current VM window dimensions (r_x, r_y) rather than absolute pixel values. This ensures automation continues to function correctly regardless of where the VM window is placed on the host display.

To move beyond static scripting, each bot incorporates a perception layer. OpenCV (Open Source Computer Vision Library) detects rectangular form controls by shape and contrast, allowing the bot to locate input fields dynamically rather than relying on hard-coded positions. Tesseract OCR extracts text from data display screens — the patient list, for example — so that records can be read and returned to the calling system without any database access. Screen-state detection confirms the bot is on the correct screen before performing sensitive operations such as patient removal.

These bots transform what is otherwise a closed, GUI-only application into a service that can receive instructions from MongoDB or respond to REST API calls. From the perspective of the legacy application, it is simply being operated by a fast, accurate user — no modification is required.

IV. Experimental Evaluation

A. Setup and Target System

The evaluation target is a Visual Basic 6 hospital patient-management application running inside Oracle VirtualBox 7.0 with Windows XP SP3, 512 MB RAM, and 1024×768 display resolution. The VM host was a Windows 11 workstation. This configuration was chosen deliberately: Windows XP on a local VM represents a realistic worst-case modernization scenario, combining an unsupported OS, an aging application framework, and a virtualized display stack that introduces additional rendering unpredictability.

Three workflows were evaluated 50 times each:

- Patient List Retrieval: navigate to the patient list screen, capture all visible records via OCR, return structured data.
- Add New Patient: navigate to the registration form, populate seven fields, submit, and verify the new record appears in the list.

Remove Patient: navigate to the patient list, open the delete dialog, inject the target patient ID, confirm deletion, and verify the record is gone.

B. Metrics

Four primary metrics were recorded per execution. Task Success Rate measures the fraction of trials that completed all steps and passed the final post-condition check. Average Execution Time is measured from API receipt of the task request to delivery of the result. OCR Field-Level Accuracy is the exact-match rate for individual fields extracted from the patient list, computed against a manually verified ground truth. Workflow Recovery Rate counts the fraction of initially failing executions that ultimately completed through retry or fallback.

C. Results and Analysis

Table I presents results across all three workflow categories. The Add New Patient workflow had the longest average execution time at 11.8 seconds, driven by seven sequential field entries and mandatory post-insertion verification. Patient List Retrieval showed the lowest success rate at 97.3%, with failures attributable to OCR column misalignment when patient names exceeded approximately 20 characters. Remove Patient achieved the highest success rate at 98.1%, reflecting its comparatively simple interaction sequence.

Table I -Evaluation Results

Workflow	Avg. Time (s)	Success	OCR Accuracy	Recovery
Patient List Retrieval	4.2 s	97.3%	94.1%	100%
Add New Patient	11.8 s	95.6%	N/A	96.4%
Remove Patient	7.4 s	98.1%	N/A	100%
Overall / Average	7.8 s	97.0%	94.1%	98.8%

Of the 27 total initial failures across 150 trials, 24 recovered automatically through the retry and fallback mechanisms. The three failures requiring human intervention were all caused by VM display rendering timeouts during screenshot capture — the legacy application repainted the screen more slowly than the configured 2-second threshold after a dialog closed. No trial required full Sagas-pattern compensatory rollback, confirming that retry and fallback strategies were sufficient to handle all observed failure modes.

One result that stood out was the contextual bandit model's observable learning behavior in SOACRS. By approximately trial 30, the model had converged on a preference for the coordinate-based interaction agent specifically for the Date of Birth field. The reason became clear on inspection: Tesseract was systematically misidentifying the date-picker widget as a plain text input, causing the vision-based agent to send raw text rather than opening the calendar control. The bandit model detected the pattern of failures on that specific field and shifted weight to the coordinate agent, correcting the behavior autonomously without human intervention

Table II compares LAMA against five representative approaches across eight capability dimensions. Conventional RPA achieves source-code independence but lacks UI generation, adaptive planning, digital-twin validation, and API exposure. UI-TARS provides strong vision-based GUI interaction but operates as a standalone agent without coordination, experience memory, or API capabilities. Only LAMA provides all eight evaluated capabilities simultaneously.

Table II - Feature Comparison with Related Approaches

Feature	Conv. RPA	Scraping	Rewrite	UI-TARS	MAS Orch.	Proposed
No source-code access needed	✓	✓	✗	✓	○	✓
Modern UI generation	✗	✗	✓	✗	✗	✓
Adaptive agent planning	✗	✗	✓	○	✓	✓
Vision-based GUI interaction	○	✗	✗	✓	✗	✓
Digital-twin validation	✗	✗	✗	✗	✗	✓
REST / GraphQL API exposure	✗	✗	✓	✗	✗	✓
Self-healing on UI drift	✗	✗	✓	○	○	✓
Semantic experience memory	✗	✗	✗	✗	✗	✓

V. Discussion

A. *Key Findings*

Achieving a 97.0% mean success rate confirms the core hypothesis: agentic AI middleware can make legacy GUI automation reliable enough for real operational use. The biggest single contributor to that reliability was the three-strategy hybrid input injection approach combining PyAutoGUI, PyDirectInput, and AutoHotkey, which reduced input failure rates from approximately 12% (single-library baseline) to under 2%.

The contextual bandit result was the most technically significant finding. The Date of Birth field self-correction behavior was not explicitly designed — it emerged from the model's normal operation over repeated trials. This is a concrete demonstration that the self-optimization mechanism works not only in theory but in a real testing environment with genuine OCR errors.

The digital-twin simulator also proved more valuable during development than anticipated. Because the twin replays scenarios against a recorded VM state, hundreds of test iterations could be run without keeping the Windows XP VM active, reducing computational overhead substantially and making it feasible to test edge cases that would have been tedious to reproduce manually against the live VM.

B. *Limitations*

Four limitations constrain immediate production applicability. First, the RPA connector depends on Windows-only libraries (PyAutoGUI, PyDirectInput, AutoHotkey), restricting the middleware host to Windows environments. Second, 94.1% OCR accuracy would be insufficient in production healthcare contexts where patient record fidelity is regulated. Third, the contextual bandit model requires approximately ten trials on a given field type before its selections converge, representing reduced performance during the initial deployment period for low-volume deployments. Fourth, the framework has not undergone formal penetration testing or regulatory compliance review for HIPAA (Health Insurance Portability and Accountability Act) or PCI-DSS (Payment Card Industry Data Security Standard).

C. *Future Work*

Priority future directions include:

- Cross-platform input injection support via xdotool (Linux) and cliclick (macOS).

- Replacement of Tesseract with a fine-tuned TableFormer model for structured legacy table extraction.
- Extension of the Frontend Builder to multi-screen workflow capture with natural language intent recognition.
- Automated regression testing of middleware updates against the digital-twin simulator.

Formal security audit and regulatory compliance gap analysis prior to regulated-environment deployment.

VI. Conclusion

This paper presented LAMA, a four-component agentic AI middleware framework that modernizes legacy enterprise applications without modifying their source code. By combining LLM-powered interface generation (Frontend Builder), BDI-inspired adaptive planning with contextual bandit delegation (SOACRS), DAG-based multi-agent orchestration with Sagas-pattern compensation (Orchestrator), and CNN-driven adaptive RPA with digital-twin validation and REST/GraphQL API exposure (RPA Connector), LAMA creates a complete, intelligent bridge between inaccessible GUI-only legacy systems and modern digital workflows.

Evaluated on a Visual Basic 6 hospital patient-management system inside a Windows XP VM — a scenario chosen to represent a realistic worst-case modernization target — the framework achieved 97.0% mean task success, 7.8 s average execution time, 94.1% OCR field accuracy, and 98.8% workflow recovery across 150 trials, with zero legacy code modification. The contextual bandit delegation mechanism demonstrated autonomous adaptation to a systematic vision-detection error, providing concrete evidence that the self-optimization component functions under real operating conditions.

LAMA offers healthcare, finance, and government organizations a practical, low-risk modernization pathway that preserves operational continuity while delivering the integration, automation, and accessibility benefits of modern platforms. The framework and evaluation datasets will be released as open-source to support reproducibility and further research in agentic legacy system modernization.

References

[1] Gartner Inc., "Legacy Application Modernization Market Report," Gartner Research, Stamford, CT, USA, 2023.

[2] K. H. Bennett, "Legacy systems: Coping with success," IEEE Software, vol. 12, no. 1, pp. 19–23, Jan. 1995.

[3] M. Lacity and L. Willcocks, Robotic Process Automation and Risk Mitigation: The Definitive Guide. Stratford-upon-Avon: SB Publishing, 2017.

[4] R. Oberhauser and A. Pogolski, "Adaptive robotic process automation," in Proc. IEEE Int. Conf. Autonomic and Intelligent Systems (AIMS), Colmar, France, 2019, pp. 29–36.

[5] R. Kazman, P. Kruchten, and K. H. Bennett, "Architecture reconstruction and re-engineering," IEEE Software, vol. 17, no. 4, pp. 49–57, Jul. 2000.

[6] D. Perez-Castillo, I. Garcia-Rodriguez, M. Piattini, F. Ruiz, and O. Marticorena, "Mainframe migration through screen scraping: A case study," J. Syst. Softw., vol. 136, pp. 64–86, Feb. 2018.

[7] P. Ribeiro, A. Pereira, and H. Santos, "RPA and AI in organizations: A systematic literature review," Bus. Process Manag. J., vol. 27, no. 6, pp. 1513–1534, 2021.

[8] M. El-Gharib, W. Awad, and A. M. Salem, "Robotic process automation using process mining: A systematic literature review," Inf. Syst. Front., 2023. doi: 10.1007/s10796-023-10373-5

[9] T. Beltramelli, "pix2code: Generating code from a GUI screenshot," in Proc. ACM SIGCHI Symp. Eng. Interactive Comput. Syst. (EICS), Paris, France, 2018, pp. 1–6.

[10] B. Li, Y. Zhou, T. Xu, X. Chen, G. Li, and K. Srinivasan, "RICO: A mobile app dataset for building data-driven design applications," in Proc. 30th ACM Symp. User Interface Softw. Technol. (UIST), Quebec City, Canada, 2017, pp. 845–854.

[11] J. Luo, H. Xu, and Z. Liu, "OmniParser: A unified framework for visual UI understanding," 2024, arXiv:2408.00203.

[12] C. Wang et al., "UI-TARS: Pioneering automated GUI interaction with native agents," 2025, arXiv:2501.12326.

[13] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," IEEE Trans. Comput., vol. C-29, no. 12, pp. 1104–1113, Dec. 1980.

[14] H. P. Nii, "The blackboard model of problem solving and the evolution of blackboard architectures," AI Magazine, vol. 7, no. 2, pp. 38–53, 1986.

[15] M. E. Bratman, Intention, Plans, and Practical Reason. Cambridge, MA: Harvard University Press, 1987.

[16] A. S. Rao and M. P. Georgeff, "Modeling rational agents within a BDI architecture," in Proc. 2nd Int. Conf. Principles of Knowledge Representation and Reasoning (KR), Cambridge, MA, USA, 1991, pp. 473–484.

[17] F. Meneguzzi and L. A. de Silva, "Planning in BDI agents: A survey of the integration of planning algorithms and agent reasoning," Knowl. Eng. Rev., vol. 30, no. 1, pp. 1–44, 2015.

[18] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in Proc. 1st Int. Conf. Multi-Agent Systems (ICMAS), San Francisco, CA, USA, 1995, pp. 312–319.

[19] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. World Wide Web (WWW), Raleigh, NC, USA, 2010, pp. 661–670.

[20] K. Lim, Y. Tan, and M. Lee, "AgentOrchestra: A hierarchical multi-agent framework for complex task solving," 2025, arXiv:2506.12508.

[21] R. Gupta and N. Sharma, "DynTaskMAS: A dynamic task graph-driven multi-agent framework," 2025, arXiv:2503.07675.

[22] H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD Int. Conf. Management of Data, San Francisco, CA, USA, 1987, pp. 249–259.

[23] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Mountain View, CA, USA, Tech. Rep., 2010. [Online]. Available: https://research.google/pubs/pub36356/

[24] M. Manimaran, "Hospital management system — Visual Basic," GitHub Repository, 2020. [Online]. Available: https://github.com/manimaran96/HospitalManagementSystemVisualBasic